

UrbanWatch

(Banco de Dados)

Este documento apresenta o banco de dados desenvolvido para o aplicativo **UrbanWatch**. O objetivo é mostrar como as informações do sistema foram organizadas e estruturadas para garantir que os dados fiquem bem armazenados e possam ser acessados de forma prática.

O banco foi construído em **PostgreSQL** e conectado à aplicação usando **Spring Boot**, permitindo que os dados sejam salvos e consultados automaticamente. As principais tabelas envolvem **usuários**, **reviews** e **imagens dos reviews**, que se relacionam entre si para representar as funcionalidades do aplicativo.

Ao longo do documento, serão mostrados os modelos de dados, os relacionamentos entre as tabelas e exemplos de como o sistema manipula essas informações. A ideia é servir como guia para entender como o UrbanWatch organiza e utiliza seu banco de dados.

O modelo conceitual do UrbanWatch foi pensado para organizar as principais informações que o aplicativo precisa armazenar. Ele define as entidades (objetos principais do sistema) e como elas se relacionam entre si.

Entidades e Relacionamentos:

1. Users → Calls

Cardinalidade: **1:N**

Um usuário pode criar vários chamados.

Cada chamado pertence a apenas um usuário.

FK: `calls.user_id` → `users.id`

2. Calls → Comments

Cardinalidade: **1:N**

Um chamado pode ter vários comentários.

Cada comentário pertence a um único chamado.

FK: `comments.call_id` → `calls.id`

3. Users → Comments

Cardinalidade: **1:N**

Um usuário pode realizar vários comentários.

Cada comentário pertence a apenas um usuário.

FK: `comments.user_id` → `users.id`

4. Comments → Votes

Cardinalidade: **1:N**

Um comentário pode receber vários votos.

Cada voto pertence a um único comentário.

FK: votes.comment_id → comments.id

5. Users → Votes

Cardinalidade: **1:N**

Um usuário pode realizar vários votos.

Cada voto pertence a apenas um usuário.

FK: votes.user_id → users.id

6. Calls → Review

Cardinalidade: **1:1**

Um chamado pode ter uma avaliação após sua finalização.

Cada avaliação está associada a um único chamado.

FK: reviews.call_id → calls.id

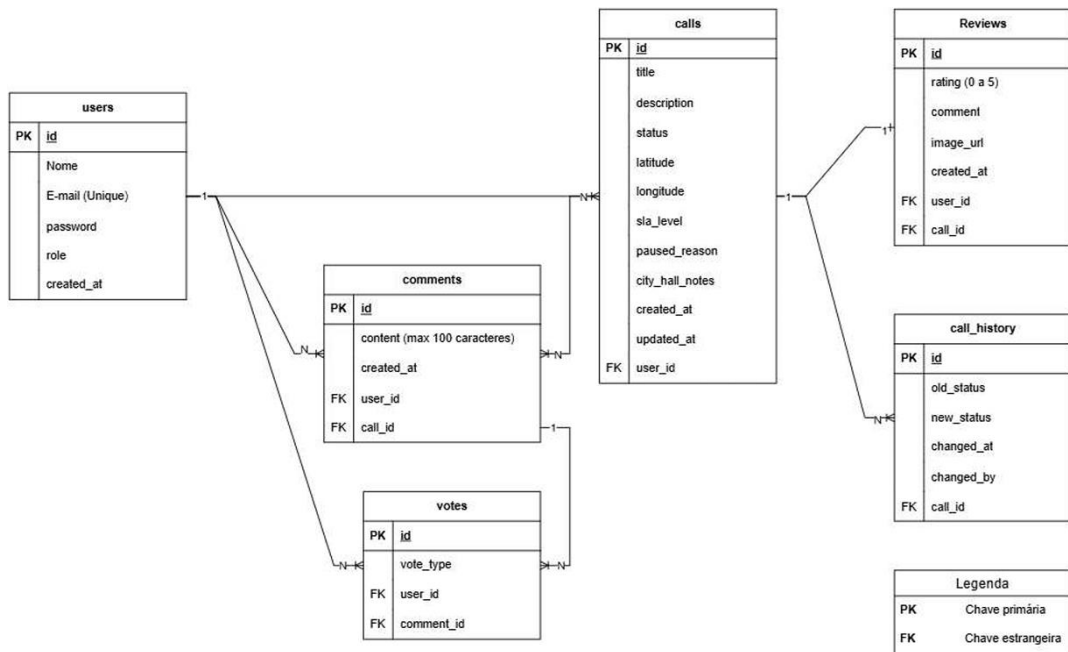
7. Calls → Call_History

Cardinalidade: **1:N**

Um chamado pode ter vários registros de histórico.

Cada registro pertence a um único chamado.

FK: call_history.call_id → calls.id



Modelo Físico:

```
CREATE TABLE users (
  id          BIGSERIAL PRIMARY KEY,
  name        VARCHAR(120) NOT NULL,
  email       VARCHAR(160) NOT NULL UNIQUE,
  password    VARCHAR(255) NOT NULL,
  role        VARCHAR(30)  NOT NULL CHECK (
    role IN ('CITIZEN', 'CITY_HALL', 'ADMIN')
  ),
  created_at  TIMESTAMP NOT NULL
);

CREATE TABLE calls (
  id          BIGSERIAL PRIMARY KEY,
  title       VARCHAR(200) NOT NULL,
  description TEXT NOT NULL,
  status      VARCHAR(30)  NOT NULL
CHECK (
  status IN (
    'PENDENTE',
    'RECEBIDO',
    'EM_AVALIACAO',
    'EM_DESLOCAMENTO',
    'EM_EXECUCAO',
    'FINALIZADO',
    'PAUSADO'
  )
),
  latitude    DOUBLE PRECISION,
  longitude   DOUBLE PRECISION,
  sla_level   VARCHAR(20) NOT NULL
CHECK (
  sla_level IN ('NORMAL', 'ATENCAO', 'CRITICO')
),
  paused      BOOLEAN NOT NULL,
  prefeitura_observation TEXT,
  created_at  TIMESTAMP NOT NULL,
  updated_at  TIMESTAMP,
  paused_at   TIMESTAMP,

  user_id     BIGINT NOT NULL,

  CONSTRAINT fk_calls_user
    FOREIGN KEY (user_id)
    REFERENCES users(id)
);
```

```
CREATE TABLE call_history (
  id          BIGSERIAL PRIMARY KEY,
  call_id     BIGINT NOT NULL,

  status_anterior VARCHAR(30) CHECK (
    status_anterior IN (
      'PENDENTE',
      'RECEBIDO',
      'EM_AVALIACAO',
      'EM_DESLOCAMENTO',
      'EM_EXECUCAO',
      'FINALIZADO',
      'PAUSADO'
    )
  ),

  status_novo   VARCHAR(30) NOT NULL CHECK (
    status_novo IN (
      'PENDENTE',
      'RECEBIDO',
      'EM_AVALIACAO',
      'EM_DESLOCAMENTO',
      'EM_EXECUCAO',
      'FINALIZADO',
      'PAUSADO'
    )
  ),

  observacao   TEXT,
  data_alteracao TIMESTAMP NOT NULL,

  CONSTRAINT fk_call_history_call
    FOREIGN KEY (call_id)
    REFERENCES calls(id)
);

CREATE TABLE call_images (
  id          BIGSERIAL PRIMARY KEY,
  file_name    VARCHAR(255) NOT NULL,
  content_type VARCHAR(255) NOT NULL,
  data        BYTEA NOT NULL,
  created_at   TIMESTAMP NOT NULL,

  call_id     BIGINT NOT NULL,

  CONSTRAINT fk_call_images_call
    FOREIGN KEY (call_id)
    REFERENCES calls(id)
);
```

```

CREATE TABLE comments (
  id          BIGSERIAL PRIMARY KEY,
  content     TEXT NOT NULL,
  created_at  TIMESTAMP NOT NULL,

  call_id     BIGINT NOT NULL,
  user_id     BIGINT NOT NULL,

  CONSTRAINT fk_comments_call
    FOREIGN KEY (call_id)
      REFERENCES calls(id),

  CONSTRAINT fk_comments_user
    FOREIGN KEY (user_id)
      REFERENCES users(id)
);

CREATE TABLE votes (
  id          BIGSERIAL PRIMARY KEY,
  vote_type   VARCHAR(10) NOT NULL CHECK (
    vote_type IN ('LIKE', 'DISLIKE')
  ),
  created_at  TIMESTAMP NOT NULL,

  user_id     BIGINT NOT NULL,
  comment_id  BIGINT NOT NULL,

  CONSTRAINT fk_votes_user
    FOREIGN KEY (user_id)
      REFERENCES users(id),

  CONSTRAINT fk_votes_comment
    FOREIGN KEY (comment_id)
      REFERENCES comments(id),

  CONSTRAINT uq_votes_user_comment
    UNIQUE (user_id, comment_id)
);

CREATE TABLE call_reviews (
  id          BIGSERIAL PRIMARY KEY,
  rating      INTEGER NOT NULL CHECK (rating
    BETWEEN 1 AND 5),
  comment     TEXT,
  created_at  TIMESTAMP NOT NULL,

  call_id     BIGINT NOT NULL,
  user_id     BIGINT NOT NULL,

  CONSTRAINT fk_call_reviews_call
    FOREIGN KEY (call_id)
      REFERENCES calls(id),

  CONSTRAINT fk_call_reviews_user
    FOREIGN KEY (user_id)
      REFERENCES users(id)
);

```

```

CREATE TABLE review_images (
  id          BIGSERIAL PRIMARY KEY,
  file_name    VARCHAR(255) NOT NULL,
  content_type VARCHAR(255) NOT NULL,
  data         BYTEA NOT NULL,
  created_at   TIMESTAMP NOT NULL,

  review_id    BIGINT NOT NULL,

  CONSTRAINT fk_review_images_review
    FOREIGN KEY (review_id)
      REFERENCES call_reviews(id)
);

```

Tabela: users

- Guarda os dados dos usuários cadastrados no UrbanWatch.
- **id**: identificador único de cada usuário.
- **name**: nome do usuário.
- **email**: endereço de e-mail (único).
- **password**: senha de acesso.
- **role**: papel do usuário no sistema (ex.: cidadão, prefeitura, admin).
- **created_at**: data de criação do cadastro.
- **Regras**: o e-mail não pode se repetir e o campo role só aceita valores pré-definidos.

Tabela: calls

- Representa os chamados abertos pelos usuários.
- **id**: identificador único do chamado.
- **title**: título do chamado.
- **description**: descrição detalhada.
- **status**: situação atual (pendente, em execução, finalizado etc.).
- **latitude e longitude**: localização do chamado.
- **sla_level**: nível de prioridade (normal, atenção, crítico).
- **paused_reason / prefeitura_observation**: observações adicionais.
- **created_at e updated_at**: datas de criação e atualização.
- **user_id**: chave estrangeira que liga o chamado ao usuário que o criou.
- **Regras**: o status e o nível de SLA só aceitam valores pré-definidos, e cada chamado pertence a um único usuário.

Tabela: call_history

- Guarda o histórico de mudanças de status de um chamado.
- **id**: identificador único do registro.
- **call_id**: chave estrangeira que liga ao chamado.
- **status_anterior**: status antes da alteração.
- **status_novo**: status depois da alteração.
- **observacao**: comentários adicionais sobre a mudança.
- **data_alteracao**: data e hora em que a mudança foi feita.
- **Regras**: tanto status_anterior quanto status_novo só aceitam valores pré-definidos (pendente, recebido, em execução, etc.).

Tabela: call_images

- Armazena as imagens enviadas junto com um chamado.
- **id:** identificador único da imagem.
- **file_name:** nome do arquivo.
- **content_type:** tipo do arquivo (ex.: jpg, png).
- **data:** conteúdo binário da imagem.
- **created_at:** data de envio da imagem.
- **call_id:** chave estrangeira que liga a imagem ao chamado correspondente.
- **Regras:** cada imagem pertence a um único chamado.

Tabela: review_images

- Armazena as imagens que os usuários enviam junto com uma avaliação (review).
- **id:** identificador único da imagem.
- **file_name:** nome do arquivo da imagem.
- **content_type:** tipo do arquivo (ex.: jpg, png).
- **data:** conteúdo binário da imagem.
- **created_at:** data em que a imagem foi adicionada.
- **review_id:** chave estrangeira que liga a imagem ao review correspondente.
- **Regras:** cada imagem pertence a um único review, garantindo integridade no relacionamento.

Procedimento de incersão:

```
-- Inserindo cidadãos
INSERT INTO users (nome, email, senha)
VALUES
('Maria Souza', 'maria@email.com', 'senha123'),
('João Pereira', 'joao@email.com', 'abc123');

-- Inserindo chamados
INSERT INTO calls (
    title, description, status, latitude, longitude, sla_level,
    paused, prefeitura_observation, created_at, updated_at, paused_at,
    user_id
)
VALUES
('Buraco na rua', 'Buraco grande na Av. Brasil', 'PENDENTE',
-20.819, -49.379, 'NORMAL',
false, NULL, NOW(), NULL, NULL, 1),

('Lâmpada queimada', 'Poste sem iluminação na praça central',
'RECEBIDO',
-20.820, -49.380, 'ATENCAO',
false, 'Será verificado pela equipe', NOW(), NULL, NULL, 5),

('Coleta de lixo atrasada', 'Caminhão não passou no bairro ontem',
'EM_AVALIACAO',
-20.821, -49.381, 'CRITICO'
true, 'Serviço pausado por manutenção', NOW(), NULL, NOW(), 6);

--Inserindo status dos chamados
INSERT INTO call_history (
    call_id, status_anterior, status_novo, observacao, data_alteracao
)
VALUES
(5, 'PENDENTE', 'RECEBIDO',
'Chamado recebido pela equipe da prefeitura', NOW()),
(5, 'RECEBIDO', 'EM_AVALIACAO', 'Equipe técnica iniciou avaliação',
NOW()),
(5, 'EM_AVALIACAO', 'EM_EXECUCAO', 'Serviço em andamento', NOW());

INSERT INTO call_images (
    file_name, content_type, data, created_at, call_id
)
VALUES (
    'teste.png',
    'image/png',
    decode('DEADBEEF', 'hex'),
    NOW(),
    6
);

--inserindo comentários
INSERT INTO comments (
    content, created_at, call_id, user_id
)
VALUES
('Esse buraco está atrapalhando muito o trânsito', NOW(), 4, 1),
('A lâmpada queimada deixa a praça perigosa à noite', NOW(), 5, 5),
('O lixo acumulado está causando mau cheiro', NOW(), 6, 6);

--Inserindo reações
INSERT INTO votes (
    vote_type, created_at, user_id, comment_id
)
VALUES
('LIKE', NOW(), 1, 4),
('DISLIKE', NOW(), 5, 5),
('LIKE', NOW(), 6, 6);
```



```

--Inserindo avaliações
INSERT INTO call_reviews (
    rating, comment, created_at, call_id, user_id
)
VALUES
(5, 'Atendimento rápido e eficiente', NOW(), 4, 1),
(3, 'Demorou um pouco, mas resolveram', NOW(), 5, 5),
(1, 'Serviço não foi concluído', NOW(), 6, 6);

--Inserindo imagens associadas às avaliações dos chamados
INSERT INTO review_images (
    file_name, content_type, data, created_at, review_id
)
VALUES (
    'avaliacao_teste.png',
    'image/png',
    decode('DEADBEEF', 'hex'),
    NOW(),
    1
);

```

Exemplo API na tabela “User”:

JSON ▼	Preview ▼
<pre> 1 { 2 "nome": "Carlos Pereira", 3 "email": "carlos.pereira@example.com", 4 "senha": "123456" 5 } </pre>	<pre> 1 { 2 "id": 4, 3 "nome": "Carlos Pereira", 4 "email": "carlos.pereira@example.com" 5 } </pre>

O banco de dados do **UrbanWatch** foi desenvolvido para organizar as principais informações do aplicativo, como usuários, chamados, comentários, votos, avaliações e históricos. A modelagem passou pelas etapas de **conceitual**, **lógica** e **física**, mostrando desde os relacionamentos entre as entidades até os comandos SQL que criam as tabelas no PostgreSQL.

Com essa estrutura, o sistema consegue registrar e relacionar os dados de forma consistente, garantindo que cada funcionalidade do UrbanWatch seja refletida corretamente no banco. O documento serve como guia para entender como as informações estão organizadas e como o banco dá suporte ao funcionamento do aplicativo.